

Discrete Codes for Low-Memory Embedding Compression in Retrieval Systems

Guillermo Pasquero

May 2, 2026

Abstract

Sentence-embedding systems traditionally store dense floating-point vectors. Under tight memory budgets (a few dozen bytes per vector), this representation becomes inefficient and discrete codes are an attractive alternative. We benchmark two well-known discrete-code families — residual vector quantization (RVQ) and product quantization (PQ) — against the standard PCA + int8 baseline on BEIR/SciFact retrieval, using the **all-MiniLM-L6-v2** sentence encoder. We refer to PCA followed by RVQ as SERA-VQ. We report mean and standard deviation across three random seeds.

At 32 bytes per embedding (a $48\times$ compression over the 1536-byte float32 representation), both discrete-code methods substantially outperform PCA + int8 in nDCG@10 (PCA + int8: 0.451; SERA-VQ: 0.554 ± 0.005 ; PQ: 0.580 ± 0.004). The advantage is consistent across budgets in the 16–32 byte range. Among discrete codes, plain PQ is competitive with or stronger than the PCA+RVQ pipeline at every byte budget tested, while using roughly $32\times$ less codebook storage.

The contribution of this work is therefore not a novel compression algorithm: PCA+RVQ is a known recipe and PQ predates it. It is an honest, seed-averaged head-to-head comparison on a concrete retrieval task that quantifies (i) the gap between discrete codes and dense linear quantization in the low-memory regime, and (ii) the relative ranking of two standard discrete-code methods.

1 Introduction

Dense vector embeddings have become a default representation for semantic similarity and information retrieval. They work well, but at scale they are expensive: a single d -dimensional float32 embedding costs $4d$ bytes, which becomes the dominant storage and bandwidth cost when serving millions of vectors.

A large body of work attacks this with dimensionality reduction (PCA), low-precision quantization (int8, fp16, binary), and discrete codes (PQ [1], RVQ, additive quantization). The compression-quality trade-off is well studied at

moderate budgets (e.g., 64–256 bytes per vector). It is less commonly reported with seed-averaged numbers and side-by-side discrete-code comparisons in the very low-memory regime — below 32 bytes per vector — which is the regime relevant to on-device retrieval and to caching layers in serving stacks.

This paper does not propose a new algorithm. It runs the comparison that the literature usually leaves implicit: at 8, 16, 24 and 32 bytes per embedding, how do PCA + int8, PCA + RVQ (which we call SERA-VQ for brevity), and PQ rank against each other and against the uncompressed baseline on a concrete BEIR retrieval task, with multiple seeds?

The findings are:

- Discrete codes (RVQ and PQ) clearly outperform PCA + int8 at the same byte budget in the 16–32 byte range. At 32 bytes, both improve nDCG@10 by roughly +0.10 over PCA + int8.
- PQ matches or beats SERA-VQ (PCA + RVQ) at every tested budget at 32 bytes the gap is +0.025 nDCG@10 in PQ’s favor.
- PQ codebooks are roughly $32\times$ smaller than RVQ codebooks at the same byte budget, since RVQ stores M full-dimensional codebooks while PQ stores M subspace codebooks.
- Even the best 32-byte configuration recovers roughly 90% of the uncompressed nDCG@10 (raw float32: 0.645). There is no free lunch — discrete codes still cost real quality.

2 Related Work

Dimensionality reduction. Principal component analysis [2] is the standard linear-compression baseline. It is often paired with low-precision storage (int8, fp16) and is what most production systems use as the first compression step.

Quantization methods. Product quantization [1] splits a k -dimensional vector into M disjoint sub-vectors of k/M dimensions and trains an independent codebook of K centroids per sub-vector. With $K = 256$, each sub-vector is encoded as one byte, so the total storage per vector is M bytes. Residual vector quantization (RVQ) instead trains M full-dimensional codebooks sequentially, each fit on the residual of the previous one. Both have a per-vector cost of M bytes when $K = 256$, but their codebook storage differs: PQ codebooks total $M \cdot K \cdot k/M = K \cdot k$ floats, while RVQ codebooks total $M \cdot K \cdot k$ floats — an $M\times$ overhead.

Sentence embeddings and retrieval benchmarks. We use Sentence-Transformers [3] embeddings (all-MiniLM-L6-v2, 384-dim) and the SciFact split of BEIR [4].

What is not new in this paper. Combining PCA with RVQ is not a novel pipeline: it is the recipe behind FAISS’s `IVF...RQ` variants and several other vector indexes. We re-run that recipe under the name SERA-VQ purely for compactness, and we make no claim of methodological novelty. The contribution is the comparison protocol, not the algorithm.

3 Method

We describe the three families of compressed representations evaluated in this paper. All operate on L_2 -normalized 384-dimensional sentence embeddings produced by `all-MiniLM-L6-v2`.

3.1 PCA + int8 (linear baseline)

We fit PCA on the corpus of document embeddings to reduce dimensionality from 384 to $k \in \{32, 64, 96, 128, 192\}$. The PCA-projected vectors are then quantized to int8 using a single global symmetric scale $s = \max(|x|)/127$, computed on documents and reused for queries to preserve cosine geometry. Memory per vector is k bytes.

3.2 SERA-VQ: PCA + RVQ

We first apply PCA to a target dimensionality k , then fit a residual vector quantizer with M codebooks of $K = 256$ centroids each. Encoding an input $z \in \mathbb{R}^k$ proceeds iteratively: at step i we select the centroid c_i in codebook $\mathcal{C}_i$ that best approximates the current residual r_{i-1} , update $r_i = r_{i-1} - c_i$, and continue. The compressed representation is the sequence of M indices, occupying M bytes. Reconstruction sums the selected centroids: $\hat{z} = \sum_i c_i$.

3.3 PQ: PCA + product quantization

We apply the same PCA reduction to dimensionality k , then split the projected vector into M disjoint sub-vectors of k/M dimensions (we only test configurations where k is a multiple of M). We fit an independent $K = 256$ codebook per sub-vector and encode each sub-vector as the index of its nearest centroid. The compressed representation is M bytes. Reconstruction concatenates the per-subspace centroids.

3.4 Codebook overhead

For all discrete-code methods the per-vector cost is M bytes, but the global codebook tables have non-trivial sizes that scale with the corpus size:

- PQ codebooks: $M \cdot K \cdot (k/M) \cdot 4 = K \cdot k \cdot 4$ bytes total. At $K = 256, k = 96$: ≈ 96 kB.

- RVQ codebooks: $M \cdot K \cdot k \cdot 4$ bytes total. At $K = 256, k = 96, M = 32$: ≈ 3 MB.

This overhead is amortized over the corpus, so it matters less for very large indexes and more for small ones. We report it explicitly because the previous version of this paper omitted it.

4 Experiments

4.1 Setup

We use the `all-MiniLM-L6-v2` sentence encoder and the test split of BEIR/SciFact (5,183 documents and 300 evaluated queries). All embeddings are L_2 -normalized before any compression step. PCA is fit on the document set; this is the same set used for retrieval evaluation, so PCA sees the test corpus (a documented limitation, not a leak relative to qrels). Codebooks (RVQ, PQ) are also fit on PCA-projected document vectors. Evaluation is brute-force cosine search; we report Recall@10 and nDCG@10.

4.2 Seeds

PCA + int8 is deterministic and is reported once. RVQ and PQ both depend on KMeans initialization; we run each configuration with three seeds (42, 123, 456) and report mean $\pm$ standard deviation of nDCG@10.

4.3 Latency

All compressed representations are reconstructed back to dense vectors before search, so search latency is essentially identical across methods at a given dimensionality (best-of-three timing: ≈ 0.04 ms/query for all methods on the 5K-document corpus, on a single CPU core). The latency differences that production systems care about — asymmetric distance computation with lookup tables — are not exercised by this protocol; this is a limitation of the comparison.

5 Results

Table 1 reports nDCG@10 for the methods that fall on the Pareto front of the byte-budget axis. Discrete codes (RVQ and PQ) dominate PCA + int8 in the 16–32 byte range. Among discrete codes, PQ matches or exceeds SERA-VQ at every byte budget tested, with a clearer margin at 24 and 32 bytes.

5.1 Trade-off vs. uncompressed embeddings

The uncompressed baseline reaches $\text{nDCG@10} = 0.6451$. The best 32-byte configuration (PQ-96) reaches 0.580, recovering 90% of uncompressed quality

Method	Bytes/vec	nDCG@10	Codebook
Raw float32	1536	0.6451	—
PCA-192 + int8	192	0.6260	—
PCA-128 + int8	128	0.6118	—
PCA-96 + int8	96	0.5957	—
PCA-64 + int8	64	0.5408	—
PQ-96 (32B)	32	0.5798 ± 0.0035	96 kB
SERA-VQ-96 (32B)	32	0.5544 ± 0.0045	3 MB
PCA-32 + int8	32	0.4510	—
PQ-96 (24B)	24	0.5385 ± 0.0050	96 kB
SERA-VQ-64 (24B)	24	0.5263 ± 0.0109	1.5 MB
PQ-96 (16B)	16	0.4895 ± 0.0051	96 kB
SERA-VQ-64 (16B)	16	0.4877 ± 0.0105	1 MB
SERA-VQ-64 (8B)	8	0.3665 ± 0.0075	0.5 MB
PQ-64 (8B)	8	0.3401 ± 0.0171	64 kB

Table 1: nDCG@10 on BEIR SciFact, mean±std over three seeds for stochastic methods. PCA + int8 is deterministic (single run). At each byte budget we show the strongest configuration of each family on the Pareto front. PQ dominates SERA-VQ at 16, 24, and 32 bytes; SERA-VQ wins at 8 bytes (where the PCA-96 PQ split has only 12 dims per codebook and underperforms the deeper-dim PCA-64 + RVQ choice).

at 1/48 of the storage. PCA-192 + int8 (192 bytes) recovers 97%. These are the two operating points worth taking seriously; configurations between them are dominated.

5.2 Codebook storage and the small-corpus caveat

For the 5K-document SciFact corpus, total storage is dominated by the codebooks for RVQ (3 MB codebooks vs. $5000 \times 32 = 160$ kB of vector codes) but not for PQ (96 kB codebooks vs. 160 kB of vector codes). On a corpus of 1M documents the picture flips entirely: vector codes dominate (32 MB vs. a few MB of codebooks for RVQ; even less for PQ). This is the regime SERA-VQ-style methods are designed for, and where the per-vector byte budget matters most.

6 Discussion and Limitations

The original version of this paper claimed that “discrete codes outperform dense embeddings in low-memory regimes” on the basis of a comparison against PCA + int8 only. That claim survives, but it is not specific to RVQ: PQ delivers the same conclusion and is the stronger of the two. Re-running the experiments with PQ as a baseline therefore changes the headline of the paper from “RVQ

wins” to “discrete codes (and specifically PQ in this regime) win”. We report the original SERA-VQ pipeline anyway because it is the recipe the original work studied, and because the gap between RVQ and PQ at fixed byte budget is itself useful information.

Concrete remaining limitations:

- **One dataset, one encoder.** BEIR/SciFact is a small (5K documents) retrieval task and `all-MiniLM-L6-v2` is a single 384-dim model. Generalization to FiQA, NFCorpus, MS MARCO and to stronger encoders (BGE, E5, Nomic, MTEB-tested models) is not established.
- **Three seeds.** Standard deviations are small (typically ≤ 0.01 nDCG@10), but three seeds is the lower end of what a peer-reviewed venue would expect.
- **No OPQ.** Optimized product quantization, which learns a rotation before splitting, is the natural next baseline and tends to close part of the gap to RVQ.
- **No binary quantization, no Matryoshka.** 1-bit-per-dimension binarization and Matryoshka-style truncation are competitive in the same byte range and are not benchmarked here.
- **Search latency under decoded vectors.** Reconstruction-based scoring discards the main practical advantage of PQ: asymmetric distance computation with lookup tables. The latency numbers in this paper therefore do not reflect production throughput.
- **PCA on the test corpus.** PCA is fit on the documents that are later retrieved. This is unsupervised so it does not leak qrels, but it does inflate absolute numbers compared to a setup where PCA is fit on a disjoint training corpus.

7 Conclusion

We compared three families of memory-efficient sentence-embedding representations on BEIR/SciFact at byte budgets between 8 and 32 bytes per vector, with multiple seeds. Discrete codes (RVQ and PQ) clearly outperform PCA + int8 in this regime; among discrete codes, PQ matches or beats RVQ at the same byte budget while using roughly $32\times$ less codebook storage. The proposed SERA-VQ pipeline therefore is not the best option in the regime it targets; PQ should be the default for low-memory retrieval at this scale. Honest reporting of this null result for SERA-VQ-vs-PQ is the main contribution of this revised version.

References

- [1] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. In *IEEE TPAMI*, 2011.
- [2] Ian Jolliffe. Principal component analysis. *Springer Series in Statistics*, 2002.
- [3] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *EMNLP*, 2019.
- [4] Nandan Thakur et al. Beir: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *NeurIPS*, 2021.